

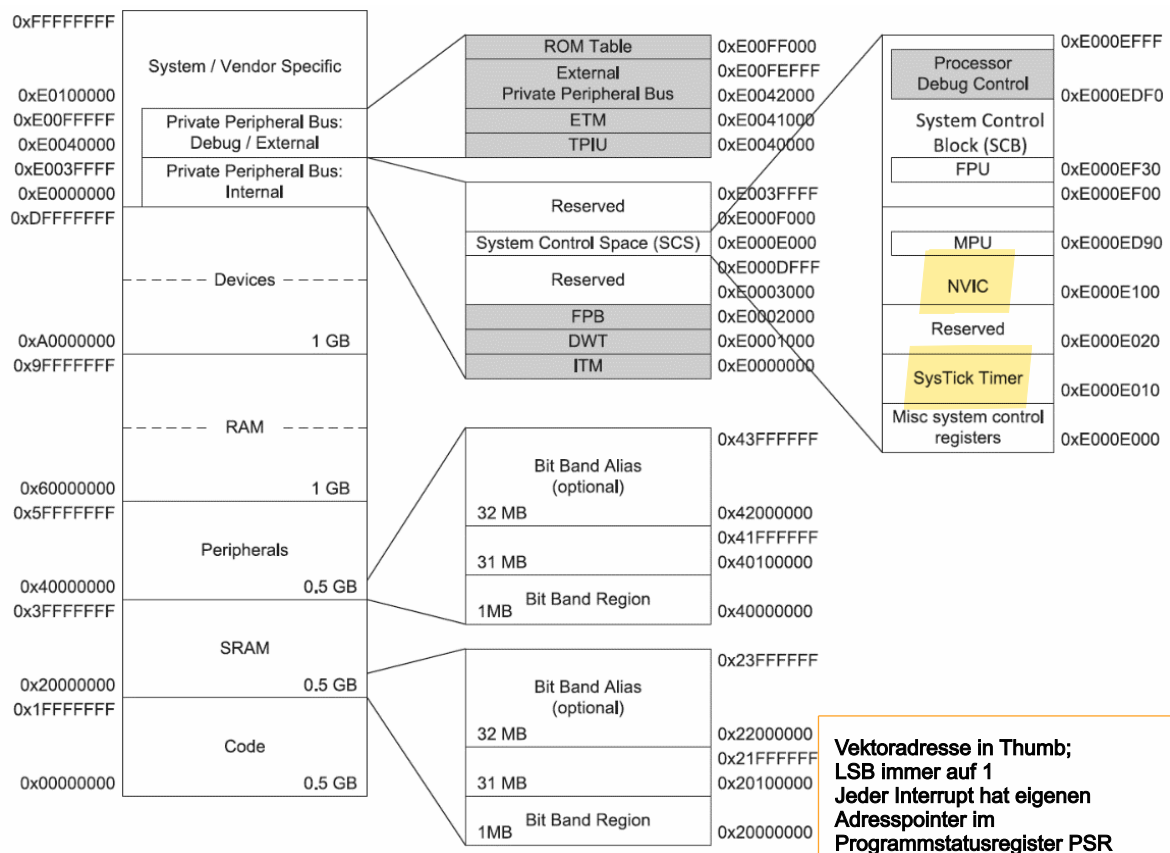
Quick Reference

Lars Kamm, Jan Wendler

1 ARM CORTEX-M MEMORY ORGANIZATION

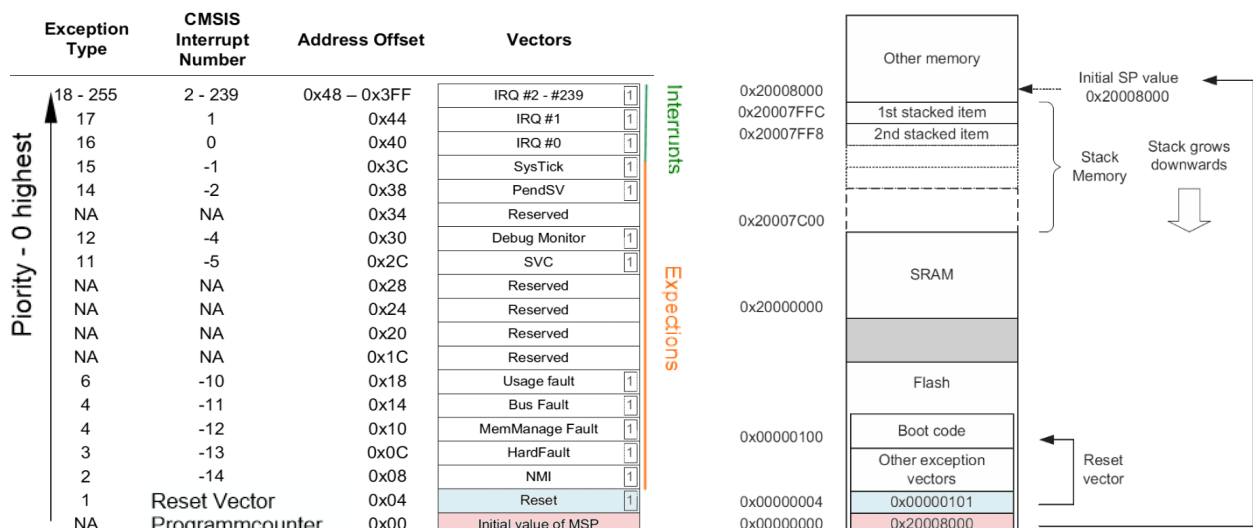
1.1 ARM Cortex-M Memory-Map

Eigenes 1.1



1.2 Vector-Table & Reset-Sequence

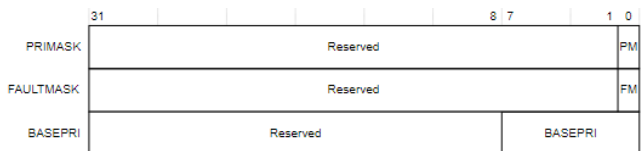
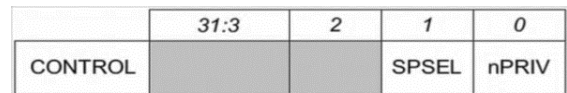
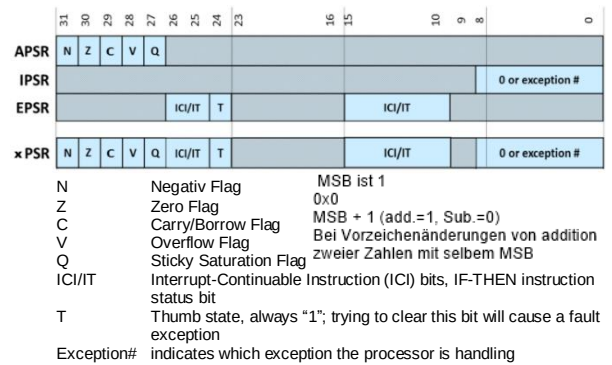
Eigenes 1.2 NVIC



1.3 Register-Set & Program Status Register

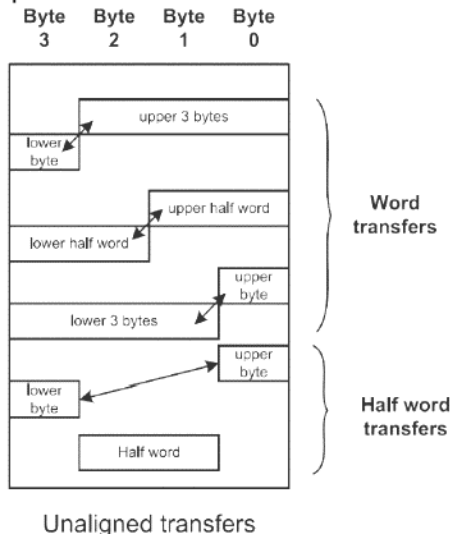
Eigenes 1.3 Spezialregister

Name	Functions (and banked registers)
R0	General purpose register
R1	General purpose register
R2	General purpose register
R3	General purpose register
R4	General purpose register
R5	General purpose register
R6	General purpose register
R7	General purpose register
R8	General purpose register
R9	General purpose register
R10	General purpose register
R11	General purpose register
R12	General purpose register
R13 (MSP)	Main Stack Pointer (MSP), Process Stack Pointer (PSP)
R14	Link Register (LR)
R15	Program Counter (PC)
xPSR	Program status registers
PRIMASK	Interrupt mask registers
FAULTMASK	Interrupt mask registers
BASEPRI	Control register
CONTROL	Control register



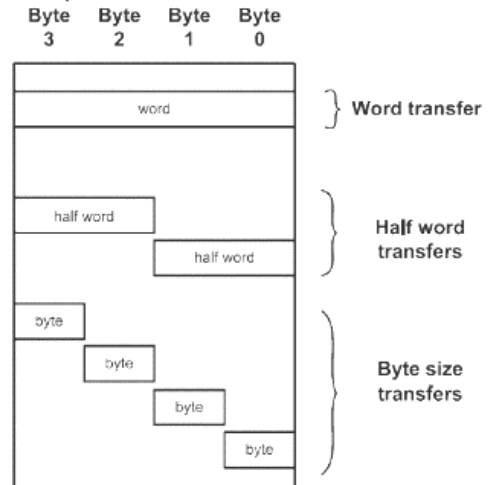
1.4 Aligment

Suboptimal -> Zusatzaufwand CPU



Unaligned transfers

Optimal: -> schnell

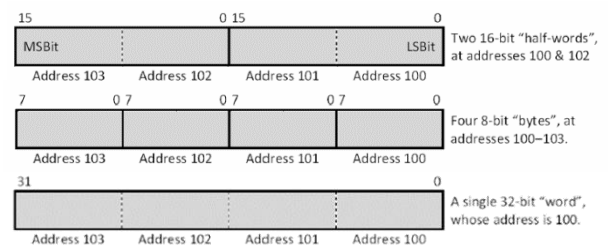


Aligned transfers

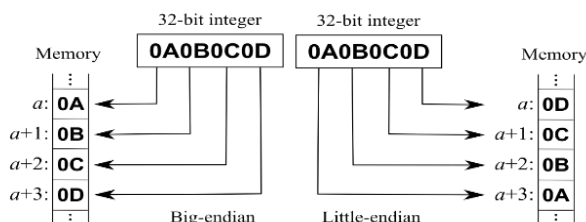
Word (32Bit): 0x0, 0x4, 0x8, 0xC

Halfword (16Bit): 0x0, 0x2, 0x4, 0x6, 0x8, 0xA, 0xC, 0xE

Byte (8Bit): 0x0, 0x1, ..., 0xF



1.5 Endianess



Endianess betrifft nur Zahlenformate die grösser als ein Byte sind (Halfword, Word, Doubleword).

Little-Endian: Least Significant Byte befindet sich an der tiefsten Adresse.

Big-Endian: Most Significant Byte befindet sich an der tiefsten Adresse.

1.6 Speichergrossen

Zur Angabe der Speicherkapazität als Anzahl von Bits, Bytes oder Codewörtern verwendet man in der Informatik in Anlehnung an die Physik die Bezeichnungen Kilo (K), Mega (M), Giga (G) und Tera (T). Diese beziehen sich auf das Binärzahlensystem:

$$1K = 2^{10} = 1'024$$

$$1M = 2^{20} = 1'024K = 1'048'576$$

$$1G = 2^{30} = 1'024M = 1'048'576K = 1'073'741'824$$

$$1T = 2^{40} = 1'024G = 1'048'576M = 1'073'741'824K = 1'099'511'627'776$$

2^5	5 -> 32
0 -> 1	6 -> 64
1 -> 2	7 -> 128
2 -> 4	8 -> 256
3 -> 8	9 -> 512
4 -> 16	10 -> 1024

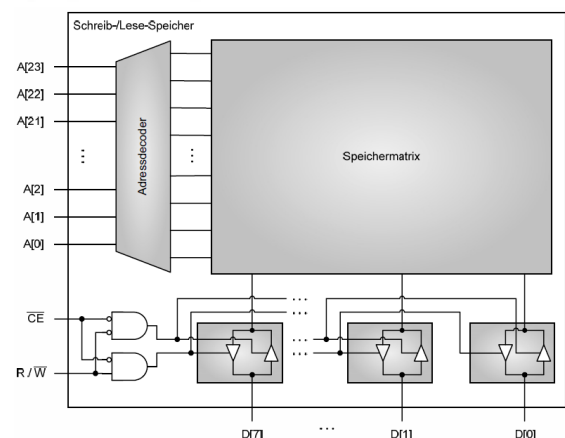
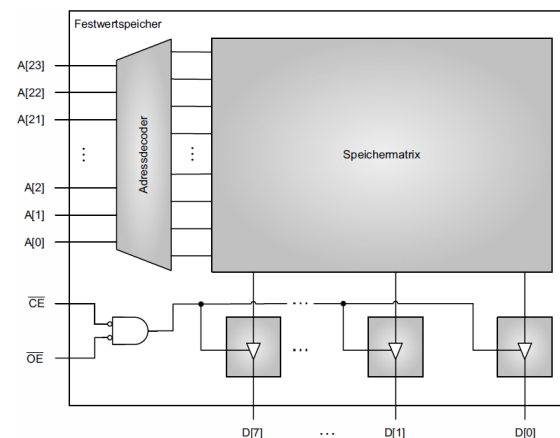
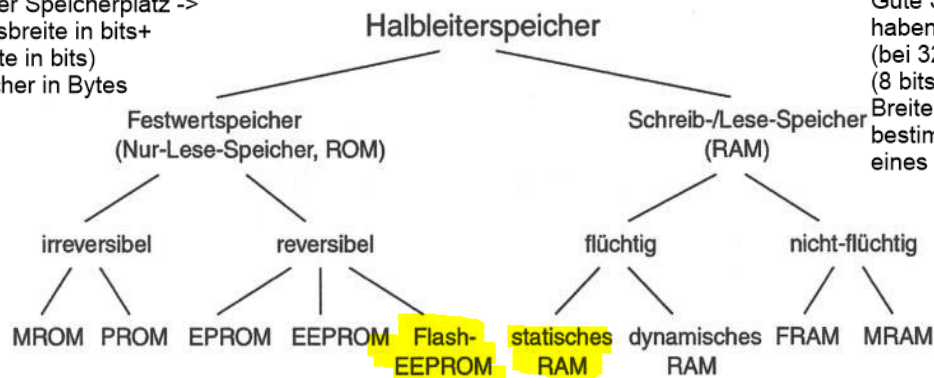
1.7 Speicherarten

Harvard = Datenbus & Adress getrennt; Von-Neumann zusammen

Eigenes 1.7 Stack

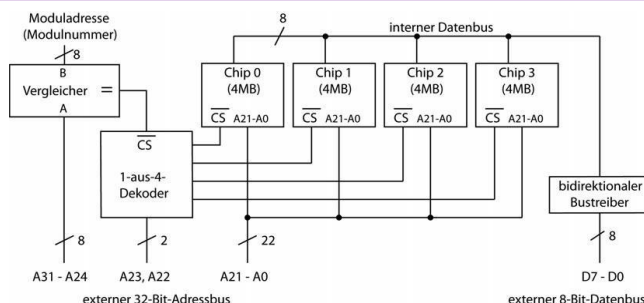
Max. möglicher Speicherplatz ->
 $2^{\text{Adressbusbreite in bits}}$
 $/ 2^{\text{Datenbusbreite in bits}}$
 = Speicher in Bytes

Gute Speicheradressen haben Endung 0,4,8,C (bei 32bit ARM) (8 bits pro Speicherplatz) Breite des Prozessors bestimmt übliche Grösse eines Speicherslots

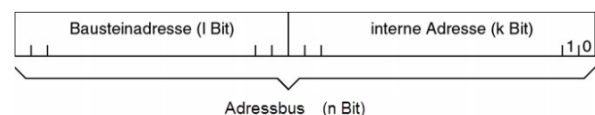


1.8 Adressdekodierung

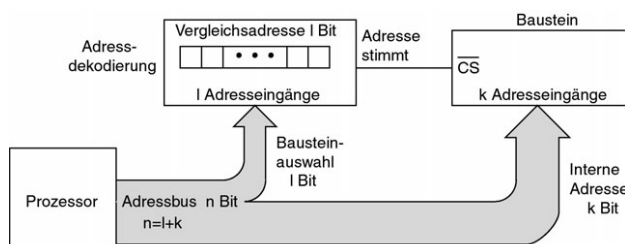
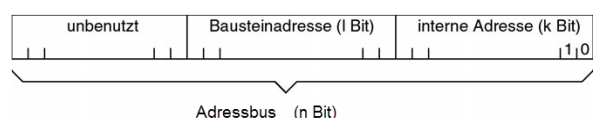
Eigenes 1.8 System & Adressbus
1.8.1 Adressierungsarten



$$k + l = n$$



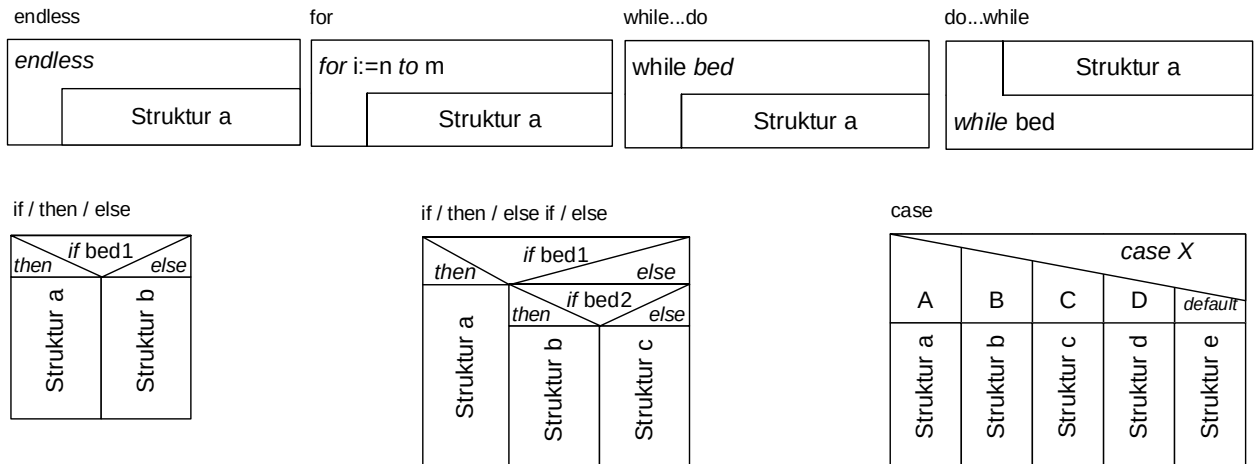
$$k + l < n$$



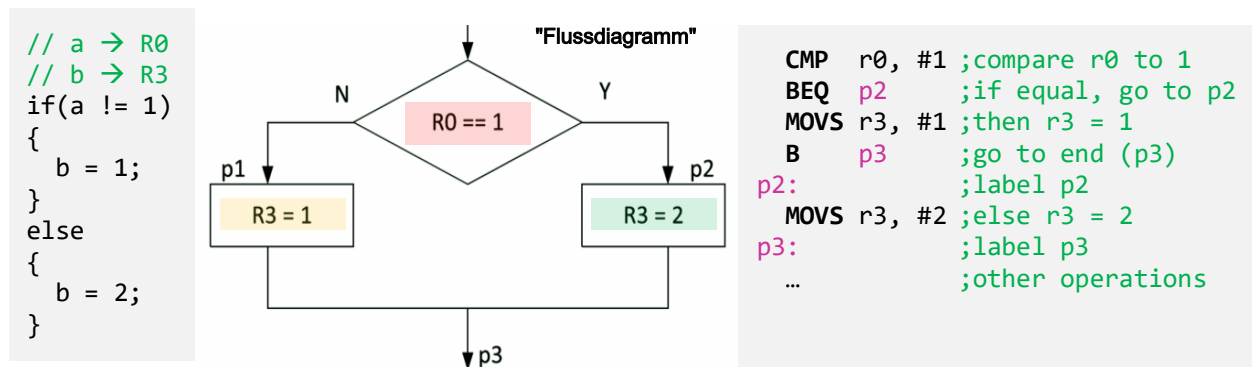
-> eigenes

2 KONTROLL- & SELEKTIONSSTRUKTUREN

2.1 Nassi Schneidermann Diagramme



2.2 Entscheidungen treffen

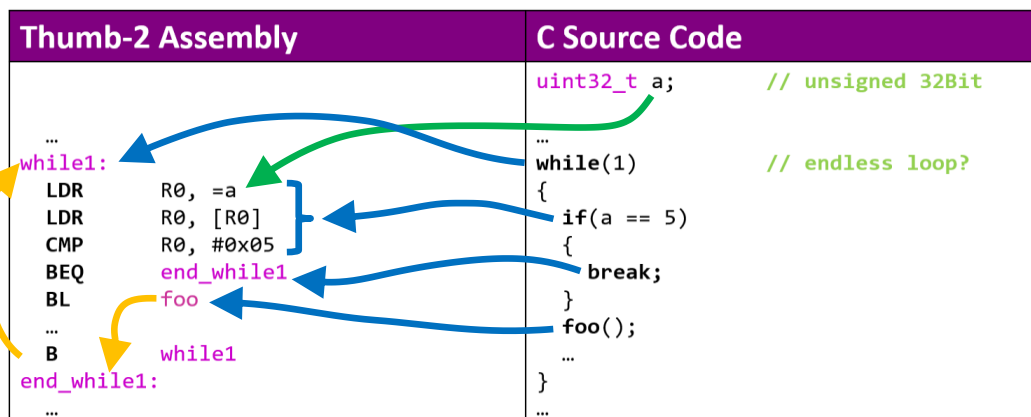


In Assembler-Sprache ist eine Entscheidung praktisch immer ein 2-stufiger Ablauf

- Benötigte Flags ermitteln**
Vergleich (**Compare**) vornehmen, indem zwei Werte voneinander subtrahiert werden. Bei der Differenzbildung ist das Resultat nicht wichtig, sondern nur dessen Eigenschaften (**Flags**).
- Zugehörige Sprünge ausführen**
Im zweiten Schritt werden mit den Flags bedingte Sprünge (**B{cond}**) ausgeführt

2.3 Umsetzung von C/C++ Strukturen

Eigenes 2.3 Compiling 7 Beispiele



3 ARM CORTEX-M0(+) INSTRUCTION-SET

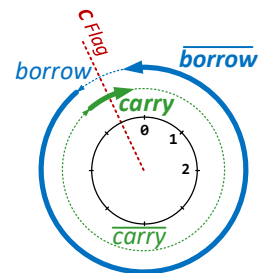
3.1 Symbole & Appendix

Symbols	Meaning
$R_d R_m R_n$	represent 32-bit registers
$\{R_d\}$	if R_d is present R_d is destination, otherwise R_n
$\{S\}$	if S is present, instruction will set condition codes
$\{cc\}$	optional logical condition, condition code suffix
$\#imm3$	any value in the range: 0...7
$\#imm5$	any value in the range: 0...31
$\#imm7$	any value in the range: 0..127
$\#imm8$	any value in the range: 0..255
$\#imm10$	any value in the range: 0..1023
$\#imm11$	any value in the range: 0..2047
label	any address within the ROM of the microcontroller; offset range relative to PC: B label -2 KB to +2 KB B<cc> label -256 bytes to +255 bytes BL label -16 MB to +16 MB
$\{reglist\}$	List of registers, sequence is not relevant

3.2 Bedingte Ausführung & Code-Zusätze {cc}

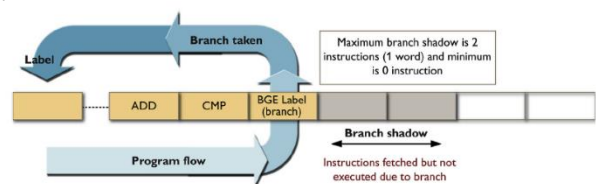
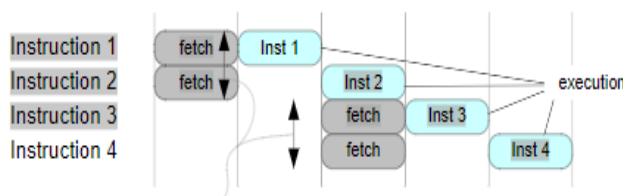
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1												

cc	Meaning	Flags (APSR) / Requirements	Binary	uint	int
EQ	Equal	Z = 1	0000	X ==	X ==
NE	Not equal	Z = 0	0001	X !=	X !=
CS or HS	Carry set, Unsigned ≥ carry OR borrow	C = 1	0010	X >=	
CC or LO	Carry clear, Unsigned < carry OR borrow	C = 0	0011	X <	
MI	Minus/negative	N = 1	0100		X -
PL	Positive or zero (non-negative)	N = 0	0101		X +
VS	Overflow	V = 1	0110		X
VC	No overflow	V = 0	0111		X
HI	Unsigned > ("Higher")	C = 1 && Z = 0	1000	X >	
LS	Unsigned ≤ ("Lower or Same")	C = 0 Z = 1	1001	X <=	
GE	Signed ≥ ("Greater than or Equal")	N = V	1010		X >=
LT	Signed < ("Less Than")	N ≠ V	1011		X <
GT	Signed > ("Greater Than")	Z = 0 && N = V	1100		X >
LE	Signed ≤ ("Less than or Equal")	Z = 1 N ≠ V	1101		X <=
AL	Always (unconditional)	any	1110		



Befehl B <cond> <label>
cond = Sprungbedingung
label = Sprungadresse ROM

3.3 Pipelining



Pipelining:

Vorteil: es wird in jedem Cycle ein Befehl ausgeführt

Nachteil: Bei Sprüngen muss die Pipeline zuerst geelert werden



3.4 ARM Cortex-M0(+) Thumb Instruktionen

Eigenes 3.4 Erläuterung

Group	Description		Syntax	Cycles	Flags				Instruction Code 16Bit																	
					N	Z	C	V	#	#	#	#	#	9	8	7	6	5	4	3	2	1	0			
Shift (immediate), add, subtract, move, and compare	Move Lo to Lo	$Rd_{0-7} \leftarrow Rm_{0-7}$	MOVS Rd, Rm	1	?	?	-	-	0	0	0	0	0	0	0	0	0	0					Rm	Rd		
	Logical shift left by immediate	$Rd_{0-7} \leftarrow Rm_{0-7} \ll \#imm5$	LSLS Rd, Rm, #<imm>	1	?	?	?	-	0	0	0	0	0					imm5				Rm	Rd			
	Logical shift right by immediate	$Rd_{0-7} \leftarrow Rm_{0-7} \gg \#imm5$	LSRS Rd, Rm, #<imm>	1	?	?	?	-	0	0	0	0	1					imm5				Rm	Rd			
	Arithmetic shift right	$Rd_{0-7} \leftarrow Rm_{0-7} \gg \#imm5$	ASRS Rd, Rm, #<imm>	1	?	?	?	-	0	0	0	1	0					imm5				Rm	Rd			
	Add 3-bit immediate	$Rd_{0-7} \leftarrow Rm_{0-7} + \#imm3$	ADDS Rd, Rm, #<imm>	1	?	?	?	?	0	0	0	1	1	1	0			imm3				Rm	Rd			
	Add All registers Lo	$Rd_{0-7} \leftarrow Rn_{0-7} + Rm_{0-7}$	ADDS Rd, Rn, Rm	1	?	?	?	?	0	0	0	1	1	0	0			Rm				Rn	Rd			
	Sub Lo and Lo	$Rd_{0-7} \leftarrow Rn_{0-7} - Rm_{0-7}$	SUBS Rd, Rn, Rm	1	?	?	?	?	0	0	0	1	1	0	1			Rm				Rn	Rd			
	Sub 3-bit immediate	$Rd_{0-7} \leftarrow Rn_{0-7} - \#imm3$	SUBS Rd, Rn, #<imm>	1	?	?	?	?	0	0	0	1	1	0	1			imm3				Rn	Rd			
	Move 8-bit immediate	$Rd_{0-7} \leftarrow \#imm8$	MOVS Rd, #<imm>	1	?	?	-	-	0	0	1	0	0					Rd					imm8			
	Compare Immediate	$Rn_{0-7} == \#imm8$	CMP Rn, #<imm>	1	?	?	?	?	0	0	1	0	1					Rn					imm8			
	Add 8-bit immediate	$Rd_{0-7} \leftarrow Rd_{0-7} + \#imm8$	ADDS Rd, #<imm>	1	?	?	?	?	0	0	1	1	0					Rd					imm8			
	Sub 8-bit immediate	$Rd_{0-7} \leftarrow Rd_{0-7} - \#imm8$	SUBS Rd, #<imm>	1	?	?	?	?	0	0	1	1	1					Rd					imm8			
Data processing	AND	$Rd_{0-7} \leftarrow Rd_{0-7} \& Rm_{0-7}$	ANDS Rd, Rm	1	?	?	-	-	0	1	0	0	0	0	0	0	0	0				Rm	Rd			
	Exclusive OR	$Rd_{0-7} \leftarrow Rd_{0-7} \wedge Rm_{0-7}$	EORS Rd, Rm	1	?	?	-	-	0	1	0	0	0	0	0	0	0	1				Rm	Rd			
	Logical shift left by register	$Rd_{0-7} \leftarrow Rd_{0-7} \ll Rm_{0-7}$	LSLS Rd, Rm	1	?	?	?	-	0	1	0	0	0	0	0	0	0	1	0			Rm	Rd			
	Logical shift right by register	$Rd_{0-7} \leftarrow Rd_{0-7} \gg Rm_{0-7}$	LSRS Rd, Rm	1	?	?	?	-	0	1	0	0	0	0	0	0	0	1	1			Rm	Rd			
	Arithmetic shift right by register	$Rd_{0-7} \leftarrow Rd_{0-7} \gg Rm_{0-7}$	ASRS Rd, Rm	1	?	?	?	-	0	1	0	0	0	0	0	0	1	0	0			Rm	Rd			
	Add with carry	$Rd_{0-7} \leftarrow Rd_{0-7} + Rm_{0-7}$	ADCS Rd, Rm	1	?	?	?	?	0	1	0	0	0	0	0	0	1	0	1			Rm	Rd			
	Sub with carry	$Rd_{0-7} \leftarrow Rd_{0-7} - Rm_{0-7}$	SBCS Rd, Rm	1	?	?	?	?	0	1	0	0	0	0	0	0	1	1	0			Rm	Rd			
	Rotate right by register	$Rd_{0-7} \leftarrow Rd_{0-7} \ggg Rm_{0-7}$	RORS Rd, Rm	1	?	?	?	-	0	1	0	0	0	0	0	0	1	1	1			Rm	Rd			
	AND test	$Rn_{0-7} \& Rm_{0-7}$	TST Rn, Rm	1	?	?	?	-	0	1	0	0	0	0	0	1	0	0	0			Rm	Rn			
	Negate	$Rd_{0-7} \leftarrow \sim(Rm_{0-7})$	RSBS Rd, Rm, #0	1	?	?	?	?	0	1	0	0	0	0	0	1	0	0	1			Rm	Rd			
	Compare Lo to Lo	$Rn_{0-7} == Rm_{0-7}$	CMP Rn, Rm	1	?	?	?	?	0	1	0	0	0	0	0	1	0	1	0			Rm	Rn			
	Compare Negative	$Rn_{0-7} == Rm_{0-7}$	CMN Rn, Rm	1	?	?	?	?	0	1	0	0	0	0	0	1	0	1	1			Rm	Rn			
	OR	$Rd_{0-7} \leftarrow Rd_{0-7} Rm_{0-7}$	ORRS Rd, Rm	1	?	?	-	-	0	1	0	0	0	0	0	1	1	0	0			Rm	Rd			
	Multiply	$Rd_{0-7} \leftarrow Rm_{0-7} * Rd_{0-7}$	MULS Rd, Rm	1 or 32 ⁽¹⁾	?	?	-	-	0	1	0	0	0	0	0	1	1	0	1			Rm	Rd			
	Bit clear	$Rd_{0-7} \leftarrow Rd_{0-7} \& \sim(Rm_{0-7})$	BICS Rd, Rm	1	?	?	-	-	0	1	0	0	0	0	0	1	1	1	0			Rm	Rd			
	Move NOT	$Rd_{0-7} \leftarrow \sim(Rm_{0-7})$	MVNS Rd, Rm	1	?	?	-	-	0	1	0	0	0	0	0	1	1	1	1			Rm	Rd			
Special data instructions and branch and exchange	Add Any to PC	$PC \leftarrow PC + Rm_{0-15}$	ADD PC, Rm	1	-	-	-	-	0	1	0	0	0	1	0	0	1				Rm	1	0	1		
	Add Any to Any	$Rd_{0-15} \leftarrow Rd_{0-15} + Rm_{0-15}$	ADD Rd, Rm	2	-	-	-	-	0	1	0	0	0	0	1	0	0				Rd			Rd		
	Address from SP and register	$Rd_{0-15} \leftarrow SP + Rd_{0-15}$	ADD Rd, SP	1	-	-	-	-	0	1	0	0	0	0	1	0	0				Rd	1	1	0	1	
	Compare Any to Any	$Rn_{0-15} == Rm_{0-15}$	CMP Rn, Rm	1	?	?	?	?	0	1	0	0	0	0	1	0	1				Rn			Rn		
	Move Any to Any	$Rd_{0-15} \leftarrow Rm_{0-15}$	MOV Rd, Rm	1	-	-	-	-	0	1	0	0	0	0	1	1	0				Rd			Rd		
	Move Any to PC	$PC \leftarrow Rm_{0-15}$	MOV PC, Rm	2	-	-	-	-	0	1	0	0	0	0	1	1	0	1				Rm	1	1	1	
	Branch With exchange	$PC \leftarrow LR$	BX Rm	2	-	-	-	-	0	1	0	0	0	0	1	1	1	0				Rm	0	0	0	
	Branch With link and exchange	$PC \leftarrow LR; LR \leftarrow retAddr$	BLX Rm	2	-	-	-	-	0	1	0	0	0	0	1	1	1	1				Rm	0	0	0	
Load from Literal Pool	Load PC-relative	$Rd \leftarrow \langle label \rangle$	LDR Rd, <label>	2 or 1 ⁽²⁾	-	-	-	-	0	1	0	0	1					Rd				imm8				
Load/store single data item	Word, register offset	$Rd \rightarrow [Rn + Rm]$	STR Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	-	-	-	0	1	0	1	0	0	0	0			Rm			Rn		Rd		
	Halfword, register offset	$Rd \rightarrow [Rn + Rm]$	STRH Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	-	-	-	0	1	0	1	0	0	0	1			Rm			Rn		Rd		
	Byte, register offset	$Rd \rightarrow [Rn + Rm]$	STRB Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	-	-	-	0	1	0	1	0	1	0			Rm			Rn		Rd			
	Signed byte, register offset	$Rd \leftarrow [Rn + Rm]$	LDRSB Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	-	-	-	0	1	0	1	0	1	1			Rm			Rn		Rd			
	Word, register offset	$Rd \leftarrow [Rn + Rm]$	LDR Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	-	-	-	0	1	0	1	1	0	0			Rm			Rn		Rd			
	Halfword, register offset	$Rd \leftarrow [Rn + Rm]$	LDRH Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	-	-	-	0	1	0	1	1	0	1			Rm			Rn		Rd			
	Byte, register offset	$Rd \leftarrow [Rn + Rm]$	LDRB Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	-	-	-	0	1	0	1	1	1	0			Rm			Rn		Rd			
	Signed halfword, register offset	$Rd \leftarrow [Rn + Rm]$	LDRSH Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	-	-	-	0	1	0	1	1	1	1			Rm			Rn		Rd			

(1) 2 if to AHB interface or SCS, 1 if to single-cycle I/O port.
(2) N is the number of elements in the list

(3) N is the number of elements in the list including PC or LR.
(4) 2 if taken, 1 if not-taken

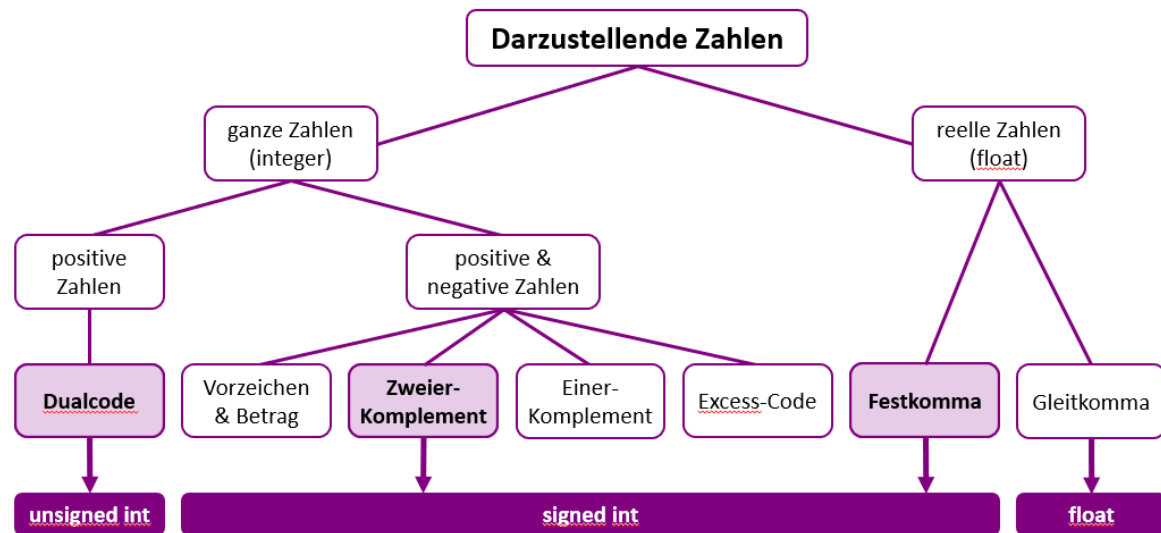


Load/store single data item	Word, immediate offset	$Rd \rightarrow [Rn + \#imm5]$	STR Rd, [Rn, #<imm>]	2 or 1 ⁽²⁾	- - - -	0 1 1 0 0	imm5	Rn	Rd
	Word, immediate offset	$Rd \leftarrow [Rn + \#imm5]$	LDR Rd, [Rn, #<imm>]	2 or 1 ⁽²⁾	- - - -	0 1 1 0 1	imm5	Rn	Rd
	Byte, immediate offset	$Rd \rightarrow [Rn + \#imm5]$	STRB Rd, [Rn, #<imm>]	2 or 1 ⁽²⁾	- - - -	0 1 1 1 0	imm5	Rn	Rd
	Byte, immediate offset	$Rd \leftarrow [Rn + \#imm5]$	LDRB Rd, [Rn, #<imm>]	2 or 1 ⁽²⁾	- - - -	0 1 1 1 1	imm5	Rn	Rd
	Halfword, immediate offset	$Rd \rightarrow [Rn + \#imm5]$	STRH Rd, [Rn, #<imm>]	2 or 1 ⁽²⁾	- - - -	1 0 0 0 0	imm5	Rn	Rd
	Halfword, immediate offset	$Rd \leftarrow [Rn + \#imm5]$	LDRH Rd, [Rn, #<imm>]	2 or 1 ⁽²⁾	- - - -	1 0 0 0 1	imm5	Rn	Rd
	SP-relative	$Rd \rightarrow [Rn + \#imm8]$	STR Rd, [SP, #<imm>]	2 or 1 ⁽²⁾	- - - -	1 0 0 1 0	Rd	imm8	
	SP-relative	$Rd \leftarrow [SP + \#imm8]$	LDR Rd, [SP, #<imm>]	2 or 1 ⁽²⁾	- - - -	1 0 0 1 1	Rd	imm8	
Generate PC-relative address	Form address from PC	$Rd \leftarrow addr[<label>]$	ADR Rd, <label>	1	- - - -	1 0 1 0 0	Rd	imm8	
Generate SP-relative address	Form address from SP	$Rd \leftarrow SP + \#imm8$	ADD Rd, SP, #<imm>	1	- - - -	1 0 1 0 1	Rd	imm8	
Miscellaneous 16-bit instructions	Add Immediate to SP	$SP \leftarrow SP + \#imm7$	ADD SP, #<imm>	1	- - - -	1 0 1 1 0 0 0 0 0	0	imm7	
	Sub Immediate from SP	$SP \leftarrow SP - \#imm7$	SUB SP, #<imm>	1	- - - -	1 0 1 1 0 0 0 0 0	1	imm7	
	Extend signed halfword to word	$Rd_{32Bit} \leftarrow signed\ Rm_{16Bit}$	SXTH Rd, Rm	1	- - - -	1 0 1 1 0 0 1 0 0 0 0	Rm	Rd	
	Extend signed byte to word	$Rd_{32Bit} \leftarrow signed\ Rm_{8Bit}$	SXTB Rd, Rm	1	- - - -	1 0 1 1 0 0 1 0 1 0 0 1	Rm	Rd	
	Extend unsigned halfword	$Rd_{32Bit} \leftarrow Rm_{16Bit}$	UXTH Rd, Rm	1	- - - -	1 0 1 1 0 0 1 0 1 0 1 0	Rm	Rd	
	Extend unsigned byte	$Rd_{32Bit} \leftarrow Rm_{8Bit}$	UXTB Rd, Rm	1	- - - -	1 0 1 1 0 0 1 0 1 0 1 1	Rm	Rd	
	Push	regs $\rightarrow mem[SP--]$	PUSH {<LoReglist>}	1+N ⁽⁴⁾	- - - -	1 0 1 1 0 1 0 0 0	lo_register_list		
	Push with link register	regs, LR $\rightarrow mem[SP--]$	PUSH {<LoReglist>, LR}	1+N ⁽³⁾	- - - -	1 0 1 1 0 1 0 1 0 1	lo_register_list		
	Disable interrupts		CPSID i	1	- - - -	1 0 1 1 0 1 1 0 1 0 0 1 1 0	0 0 1 1 0	0 0 1 0	
	Enable interrupts		CPSIE i	1	- - - -	1 0 1 1 0 1 1 0 1 0 0 1 1 1	0 0 1 1 1	0 0 1 0	
	Bytes in word	$Rd \leftarrow RevByteOrder[Rm]$	REV Rd, Rm	1	- - - -	1 0 1 1 1 0 1 0 1 0 0 0	Rm	Rd	
	Bytes in both halfwords	$Rd \leftarrow RevHalfWords[Rm]$	REV16 Rd, Rm	1	- - - -	1 0 1 1 1 0 1 0 1 0 0 1	Rm	Rd	
	Signed bottom half word	$Rd \leftarrow RevLoHalf[Rm]$	REVSH Rd, Rm	1	- - - -	1 0 1 1 1 0 1 0 1 0 1 1	Rm	Rd	
	Pop	regs $\leftarrow mem[SP++]$	POP {<LoReglist>}	1+N ⁽³⁾	- - - -	1 0 1 1 1 1 0 0 0	lo_register_list		
	Pop and return	regs, PC $\leftarrow mem[SP++]$	POP {<LoReglist>, PC}	3+N ⁽⁴⁾	- - - -	1 0 1 1 1 1 1 0 1	lo_register_list		
	Breakpoint		BKPT #<imm>	- ⁽⁶⁾	- - - -	1 0 1 1 1 1 1 1 1 0	imm8		
	Send event		SEV	1	- - - -	1 0 1 1 1 1 1 1 1 1	0 1 0 0	0 0 0 0	
	Wait for event		WFE	2 ⁽⁷⁾	- - - -	1 0 1 1 1 1 1 1 1 1	0 0 1 0	0 0 0 0	
	Wait for interrupt		WFI	2 ⁽⁷⁾	- - - -	1 0 1 1 1 1 1 1 1 1	0 0 1 1	0 0 0 0	
	Yield		YIELD	1 ⁽⁸⁾	- - - -	1 0 1 1 1 1 1 1 1 1	0 0 0 1	0 0 0 0	
No operation		NOP	1	- - - -	1 0 1 1 1 1 1 1 1 1	0 0 0 0	0 0 0 0		
Store multiple registers	Store Multiple	regs $\rightarrow mem[Rn++]$	STM Rn!, {<LoReglist>}	1+N ⁽³⁾	- - - -	1 1 0 0 0	Rn	lo_register_list	
Load multiple registers	Load Multiple, excluding base	regs $\leftarrow mem[Rn++]$	LDM Rn!, {<LoReglist>}	1+N ⁽³⁾	- - - -	1 1 0 0 1	Rn	lo_register_list	
	Load Multiple, including base	regs $\leftarrow mem[Rn]$	LDM Rn, {<LoReglist>}	1+N ⁽³⁾	- - - -	1 1 0 0 1	Rn	lo_register_list	
Conditional branch, and Supervisor Call	Conditional	if <cc> then PC $\leftarrow$ label	B<cc> <label>	1 or 2 ⁽⁵⁾	- - - -	1 1 0 1	cond	imm8	
	Supervisor Call		SVC #<imm>	- ⁽⁶⁾	- - - -	1 1 0 1 1 1 1 1 1	imm8		
Unconditional Branch	Unconditional	PC $\leftarrow$ label	B<label>	2	- - - -	1 1 1 0 0	imm11		
Branch & Link 32Bit	With link	[15 - 0]	BL<label>	3	- - - -	1 1 1 1 0	S	imm10	
		[31 - 16]			- - - -	1 1 1 1 1	J2	imm11	
Write and Read Special Register 32 Bit	Read special register	[15 - 0]	MRS Rd, <specreg>	3	- - - -	1 1 1 1 0 0 1 1 1	1 1 1 1 0	1 1 1 1	
		[31 - 16]			- - - -	1 0 0 0	Rd	SYSm	
	Write special register	[15 - 0]	MSR <specreg>, Rn	3	- - - -	1 1 1 1 0 0 1 1	1 0 0 0	Rn	
		[31 - 16]			- - - -	1 0 0 0 1 0 0 0	SYSm		
Synchronisation 32 Bit	Instruction synchronization barrier	[15 - 0]	ISB	3	- - - -	1 1 1 1 0 0 1 1 1	1 0 1 1	1 1 1 1	
		[31 - 16]			- - - -	1 0 0 0 1 1 1 1	0 1 1 0	option	
	Data memory barrier	[15 - 0]	DMB	3	- - - -	1 1 1 1 0 0 1 1	1 0 1 1	1 1 1 1	
		[31 - 16]			- - - -	1 0 0 0 1 1 1 1	0 1 0 1	option	
	Data synchronization barrier	[15 - 0]	DSB	3	- - - -	1 1 1 1 0 0 1 1	1 0 1 1	1 1 1 1	
		[31 - 16]			- - - -	1 0 0 0 1 1 1 1	0 1 0 0	option	

(5) Cycle count depends on processor and debug configuration.
(6) Excludes time spent waiting for an interrupt or event.

(7) Executes as NOP.
(8) Depends on multiplier implementation.

4 ZAHLENSYSTEME



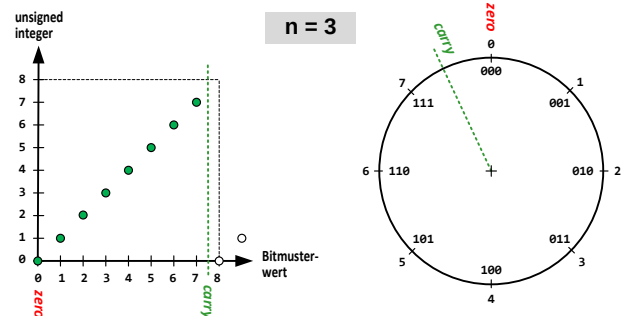
4.1 Vorzeichenlose Zahlen

Dualcode (unsigned int)

$$W = \sum_{i=0}^{n-1} d_i \cdot 2^i$$

$d_{n-1} \quad d_{n-2} \quad \dots \quad d_2 \quad d_1 \quad d_0$

Relevante Flags: Zero, Carry/Borrow



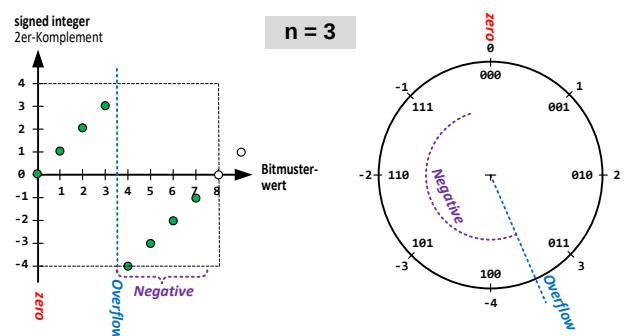
4.2 Vorzeichenbehaftete Zahlen

Zweierkomplement (signed int)

$$W = \left(\sum_{i=0}^{n-2} d_i \cdot 2^i \right) - d_{n-1} \cdot 2^{n-1}$$

$d_{n-1} \quad d_{n-2} \quad \dots \quad d_2 \quad d_1 \quad d_0$

Relevante Flag: Zero, Negative, Overflow



4.3 Zweierkomplement-Negierung

Vorgehensweise:

- 1) Vorzeichenbehaftete Zahl in Binärcode darstellen
- 2) Alle Bits einzeln invertieren
- 3) Binäre Addition mit dem Wert +1
- 4) Resultat entspricht der negierten Vorzeichenbehaftete Zahl

Beispiel: $-3_{10} = 101_2$
Invertieren: 010_2
+1: 001_2

 011_2
=====

Resultat: $+3_{10}$

4.4 Festkomma-Zahlen

Vorzeichenlose Fixed-Point-Zahlen

Allgemeine Formel (Dualcode als Basis)

$$W = \sum_{i=-m}^{n-1} d_i \cdot 2^i$$

$d_{n-1} \ d_{n-2} \ \dots \ d_1 \ d_0 \ . \ d_{-1} \ d_{-2} \ d_{-m}$

Wertebereich: $0.0 \dots 2^n - 2^{-m}$

Schrittweite: $\Delta X = 2^{-m}$

Max. Diskretisierungsfehler: $\frac{\Delta X}{2} = 2^{-(m+1)}$

Vorzeichenbehaftete Fixed-Point-Zahlen

Allgemeine Formel (2er-Komplement als Basis)

$$W = \left(\sum_{i=-m}^{n-1} d_i \cdot 2^i \right) - d_{n-1} \cdot 2^{n-1}$$

$d_{n-1} \ d_{n-2} \ \dots \ d_1 \ d_0 \ . \ d_{-1} \ d_{-2} \ d_{-m}$

Wertebereich: $-2^{n-1} \dots 2^{n-1} - 2^{-m}$

Schrittweite: $\Delta X = 2^{-m}$

Max. Diskretisierungsfehler: $\frac{\Delta X}{2} = 2^{-(m+1)}$

4.5 IQ-Zahlenformate

Integer/Quotient-Darstellung

Signed Fixed-Point mit $n = I$ und $m = Q$.

$$W = -d_{I-1} \cdot 2^{I-1} + \sum_{k=0}^{I-2} d_k \cdot 2^k + \sum_{m=1}^Q d_{-m} \cdot 2^{-m}$$

$d_{I-1} \ d_{I-2} \ \dots \ d_1 \ d_0 \ . \ d_{-1} \ d_{-2} \ d_{-Q}$
 - $I = n$ Vorkomma, $Q = m$ Nachkomma
 - Fixkomma
 - Add + Sub wie normal

Fraktionale Zahlen (oder Spezialfall I1Qx)

$$W = -d_0 + \sum_{i=1}^{n-1} d_{-i} \cdot 2^{-i}$$

MSB gibt Vorzeichen an
Häufig für % verwendet

$d_0 \ . \ d_{-1} \ d_{-2} \ \dots \ d_{-(n-1)}$

Wertebereich: $-1.0 \leq W < +1.0$

Vorzeichen: Koeffizient d_0

Genauigkeit: $2^{-(n-1)}$

4.6 Zahlenkreise

Addition: Zahlenvektoren hintereinander reihen (Anfang auf Ende)

Subtraktion: Zahlenvektoren stumpf gegeneinanderstellen (Ende auf Ende)

Unsigned Integer

Relevante Flags: **Zero, Carry/Borrow**

Zahlenvektoren immer Uhrzeigersinn

Signed Integer

Relevante Flags: **Negative, Overflow, Zero**

positiv → Zahlenvektor Uhrzeigersinn

negativ → Zahlenvektor Gegenuhrzeiger

Z Zero: wenn Ergebnis 0 = 1 ansonsten 0

N Negative: MSB = 1 → 1

C Carry:

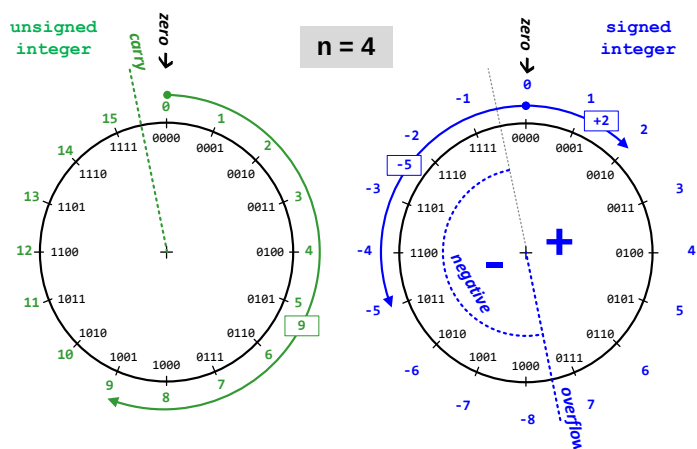
Addition: Übertrag → bei 1=1 0=0

Subtraktion: Übertrag → bei 0=1 1=0

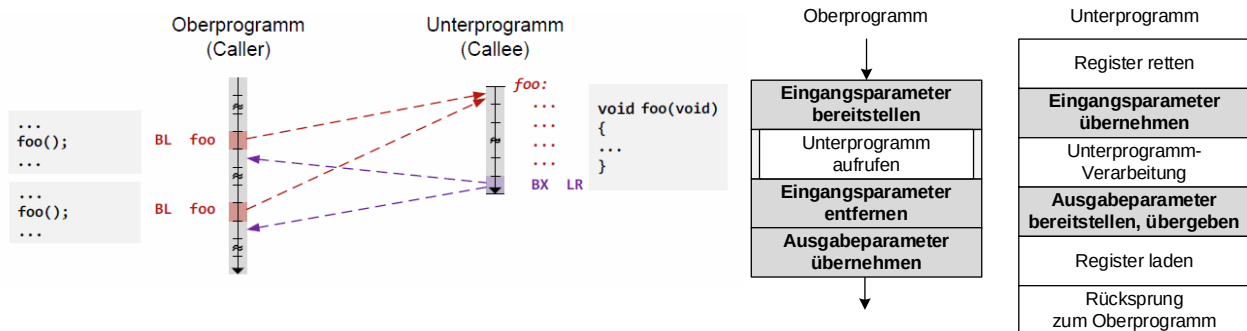
V Overflow: Zahl ist grösser als verfügbarer Speicherplatz → 1

(eingezeichnete Pfeile gehen über Grenze; $(-x)+(-x)=+$?

$(-x)-(+x)=+?$ → 1



5 SUBROUTINEN



Regeln zum ARM Architecture Procedure Call Standard (AAPCS)

- Die Register **R0-R3** werden als Parameter verwendet.
- Weitere Parameter werden auf den **Stack** gelegt.
- Der Rückgabewert ($\leq 32\text{Bit}$) wird in Register **R0** übertragen.
- Ein 64Bit-Rückgabewert wird in den Registern **R0** und **R1** übertragen.
- Funktionen müssen die Registerinhalte **R4-R11** während der Ausführung sichern und rekonstruieren.
- Die Register **R0-R3** und **R12** dürf(t)en in der Subroutine ohne Sicherung verändert werden.
- Es wird stets ein **8-Byte Alignment** auf dem Stack eingehalten.

Subroutine/ Interrupt:
Subroutine wird bewusst durch den Programmierer aufgerufen, Interrupt tritt zufällig auf & kann kein Values aufnehmen und zurückgeben

Register	Verwendungszweck	Scratch	Sichern
R0	Erster Funktionsparameter, Return -Wert (8-/16-/32-Bit)	Ja	
R1	Zweiter Funktionsparameter, Upper Word Return-Wert (64-Bit)	Ja	
R2, R3	Dritter und vierter Funktionsparameter	Ja	
R4-R7	Registervariablen (temporär)	Ja	Ja
R8-R11	Compilerabhängige Verwendung	Ja	Ja
R12	Funktionsinternes temporäres Speicherregister für Sprünge	Ja	
R13	Stack-Pointer (SP, Top-of-Stack)	Diese Register werden für spezielle Zwecke verwendet	
R14	Link Register (LR)		
R15	Programm Counter (PC)		

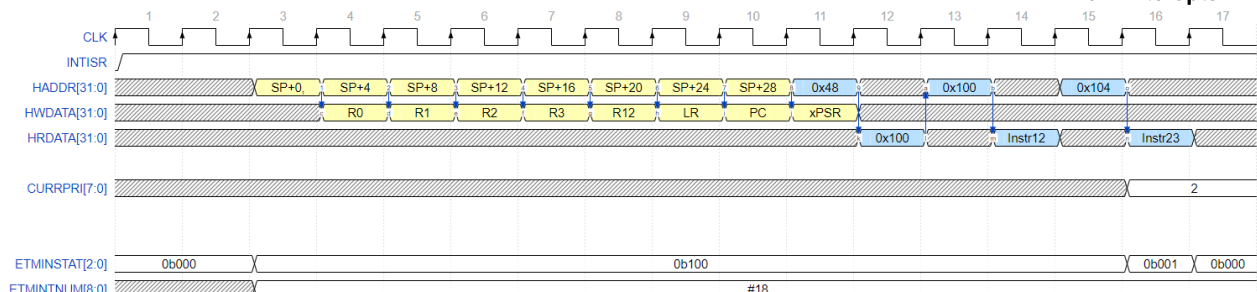
Begriffe: Calle (Code der Funktion); Caller (Adresse, Sprung in/ aus foo())

6 INTERRUPTS & EXCEPTIONS

6.1 Vector Fetch & Stacking

Eigenes 1.2 NVIC

6.1 Interrupts



- Request:** Pipeline «Execute» noch ausführen und der Pending Status setzen
- Stacking & Vector-Fetch:** Register auf Stack sichern und Sprung via Vektortabelle ausführen und Active Status setzen. **Siehe Eigenes 1.2 NVIC**
- ISR Handler:** Benutzercode ausführen und am Ende den Active Status zurücksetzen
- Unstacking:** Register wieder herstellen, Stack abbauen und Rücksprung mittels LR

- 1.1 Systemkomponenten**
System Control Space (SPC):
 - übernimmt Speicherung der Komponenten
 - Speicherung der Flags
 - Einstellungen NVIC

System Control Register:
Fault Mechanismus:
 - schnelle Reaktion auf Fehler
 - einleiten von Notfallszenarien
HardFaults:
 - immer aktiviert
 - Bus, Usage & Memory Management Fault

Wake-up Interrupt:
 - Hierarchische Abschaltung von Komponenten
 - geringerer Stromverbrauch
 - siehe Modi

Systick: (System Tick Timer)
 - 24- Bit Counter der Clocks
 - Zählt
 - Konstante Zeitbasis für Programmabläufe

Memory Protection Unit (MPU)
 - löst Exception bei unerlaubtem Speicherzugriff aus

Debug Unit:
 - Stellt 4 Breakpoints zur Verfügung
 - Programm Start, Stopp, Pause, Step
 - Data Watchpoint

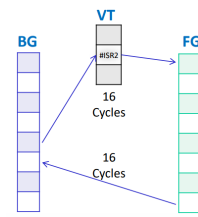
NVIC
 -> siehe NVIC

- 1.2 Nested Vector Interrupt Controller (NVIC):**
 - Background Programm wird von PC abgearbeitet bis Interrupt (ISR)
 - 1. Schritt Fetching, die Adresse wird gerufen
 - 2. Schritt Stacking, r0-r3, r12, LR, PC und PSR werden gespeichert (M0+ 15, M0 16 Cycles)
 - ISR, Benutzercode wird aufgerufen
 - Unstacking -> den jeweiligen Registern wird vorherige Wert wieder zugewiesen & Prozessor Status wird wiederhergestellt

Modi Wake-UP
Sleep- & Debug- Mode
 - unterbrochen durch Interrupts, Exceptions, Debug Events
 - Sleep on Exit nach verlassen des ISR
 - Debug Support Infos über Nutzung
Polling- Mode
 - normaler Modus, 100% CPU Auslastung

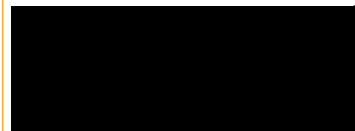
NVIC Settings
Preemption (priority):
 NVIC_SetPriority (IRQn, Priority)
Primask:
 1bit-Register, on/ off aller Interrupts
Faultmask:
 1bit-Register, on/ off alle Interrupts & Exceptions
Basepri:
 on/ off aller tiefen Interrupts

`uint8_t __get_PRIMASK(void)`
`void __set_PRIMASK(uint8_t n)`



Mögliche Interrupts:
 - Low Random : z.B. Taster
 - Low Periodic : z.B. Sys-Tick (1ms)
 - High Random : serielle Schnittstelle
 - High Periodic : ADC Sequenz (>1MHz)

- 1.7 Stack:**
 - Wächst zur tieferen Adresse
 - Temporäre Zwischenspeicherung
 - Speicher von lokalen Variablen (2x schneller) (kein laden des RAM/ROM Pointer)
 - Erhält Registerwerte bei Exceptions/ Interrupts
 - Stack Pointer SP zeigt immer auf "top of Stack"
 - Niemals unter SP lesen oder schreiben



Systembus:
 - Prozessorsysteme (RAM, ROM)
 - hohe Taktfrequenz (Hochgeschwindigkeit)
Peripheriebus:
 - Verbindung von Peripherie
AB Standard:
 - standardisierte Busanbindung
 - immer Systembus & Peripheriebus

Systembus:
AMBA AHB-lite Standard:
 - einfache Busverbindung
 - Systembus aufgeteilt in schreiben und lesen
 - Ein Zugriff dauert mind. 2 Cycles
 - Die Adresse muss vor dem lesen/ schreiben gesetzt werden

1.9 Compiling
 Der Präprozessor bereitet das Quellprogramm als Eingabe für den Compiler vor.
 Der Compiler übersetzt das Programm von einer Hochsprache in das äquivalente Assembly-Programm.
 Der Binder/Linker fasst verschiedene Dateien mit verschiebbarem Maschinencode zu einem Programm zusammen.
 Der Loader wandelt die verschiebbaren Adressen des Linkers in absolute Adressen um und lädt das Programm in den Speicher des Zielsystems.

- Compiling Schritte:**
 1. Lexikalische Analyse (entfernen Leerzeichen, Gruppieren)
 2. Syntaxanalyse (Symbole werden zusammengefasst)
 3. Semantische Analyse (Fehlerüberprüfung z.B. Typfehler)
 4. Zwischencode
 5. Code Optimierung
 6. Code Erzeugung

- 1.8.1 Adressierungsarten:**
 - Register Direkt (Register zu Register)
 - Register Indirekt (Adresse auf Register)
 - Register Indirekt mit Offset (Adresse +/-)
 - Register Indirekt mit Index (Adresse mit Offset auf anderem Register)
 - PC Relativ
 - SP Relativ

3.4 Häufige Befehle Thumb 2:
Store/Load (Verarbeitung)
Mov Datentransfer zwischen Register
LDR Register <- RAM/ROM
LDRSH Signed Halfword LDR
STR Register -> RAM/ROM
STM Rn! Speichern von Arrays
LDM Rn! Laden von Arrays
PUSH auf Stack hinzufügen (32 Bit)
POP auf Stack entfernen (32 Bit)
-{s} Endung, Setze Flags neu

Bitweise Befehle: (Verarbeitung)
ANDS Rd, Rm Rd&=Rm
ORRS Rd, Rm Rd|=Rm
BICS Rd, Rm Rd= Rd & (~Rm)
EORS Rd, Rm Rd ^=Rm (x-or)

Testen & vergleichen: (Spezial)
CMP SUBS ohne Resultat, Flags
CMN ADDS ohne Resultat, Flags

Schiebe- & Rotation (Manipulation)
LSLS Left Shift
LSRS Right Shift
ASRS Arithmetic Shift Right
RORS Rotate Shift Right

Mathe: (Verarbeitung)
ADDS Add
SUBS Sub
MULS Multiply

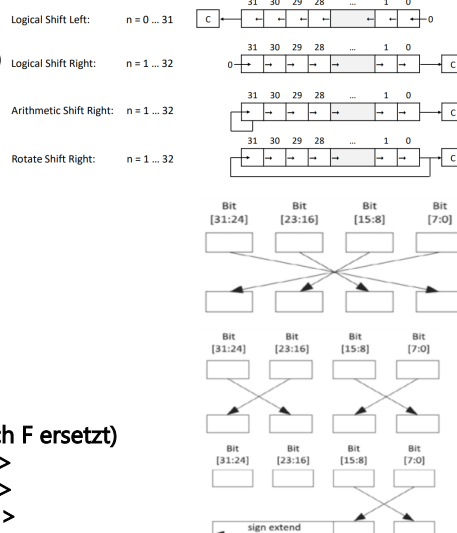
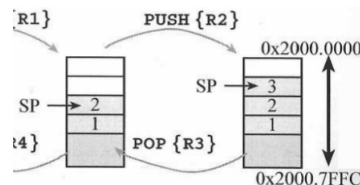
Extensions: (Manipulation)
SXTB Signed 4 Byte -> 1Byte
UXTH Unsigned 4 Byte -> 2Byte
 (Wird von MSB her gelöscht & durch F ersetzt)
REV Wechseln Byte gem. oben ->
REV16 Wechseln Byte gem. mitte ->
REVSH Wechseln Byte gem. unten ->

Befehlsarten:
Speicherzugriff: (70% der Zeit)
 - Zugriffe auf ROM, RAM, Stack

Datenverarbeitung & Manipulation:
 - logische & Arithmetische Operationen
 - Schiebe- & Rotationsoperationen
 - Konvertierungsoperationen

Programmfluss Steuerung
 - Sprungbefehle (Branch)

Spezialfunktionen
 - Interrupt- & Synchronisationsbefehle



Antworten Einstiegsquiz

Moore's Law: Komplexität (Transistoranzahl) verdoppelt alle 12-24 Monate
 Von-Neumann Systembus von Daten und Programm zusammen
 Harvard-Architektur getrennt
 Unterschied μC & μP : Alle Komponenten Rechnersystem gleichen Chip.
 Teilbussen Systembus: Adressbus (gibt Speicheradresse an) Datenbus (Übermittelt Speicherdaten) Steuerbus (Fehlerfreie Übertragung Daten)
 RISC: Reduced Instruction Set Controller (Einfacher Instruktionssatz, viele Register)
 CISC: Complex Instruction Set Controller (Komplexer Instruktionssatz, wenig Register)
 Infinite Loop: Abarbeitung Befehl in Teilschritten Fetch, Decode & Execute, Loop unendlich wiederholt
 Anzahl Bits eines Word: 32Bit- μP -> 32Bit, 16Bit- μP -> 16Bit, 8Bit- μP -> 16Bit
 Stack-Zugriff schneller Pointer (SP) gegeben, RAM-Zugriff muss Pointer geladen werden.
 Schrittweite: 2^m ; Quantisierungsfehler: $2^{-(m+1)}$
 ARM Cortex Familien: A = Application; R = Real-Time; M = Mikrocontroller
 Cortex-M4 Features gegenüber M0: DSP-Extensions, Floating Point Unit, Harvard-Architektur
 Vorteile Thumb-2 Instruktionssatz: 16Bit- und 32Bit-Befehle abarbeiten, weniger Speicherplatz, schnelle komplexe Rechenoperationen
 Cortex-M0+ Features gegenüber M0: 2-stufige Pipeline, MPU, Single-Cycle GPIO's, Micro Trace Buffer
 Pipelining: Vorteile: jedem Cycle Befehl ausgeführt; Nachteile: Sprünge Pipeline leeren
 Komponenten System Control Space: Processor Debug Control, MPU, NVIC, SysTick, System Control Register
 Unterschiedliche Addressing-Modi M0(+): Register direkt : Register indirekt (mit offset oder Index) : PC, SP relativ
 LDRSH R2, [R3,#2]: Lädt lower SH von R3+2 in Register R2 (Register indirect mit offset)
 STRB R2, [R3,R1]: Speichert das LSB von R2 in R3+R1
 Masch-code -> ADD SP, #65: 0xB041 Branch Zahlenformatbeispiele:
 BHS -> unsigned, BNE -> signed + unsigned
 Eigenschaften -> for(i=10; i>0; i--): abweisende Iteration, temp index variable mit startwert, passt Wert nach cmp an.
 Flags bei signed bzw unsigned: signed -> z,n,v, unsigned -> z,c
 Regeln für Verwendung von Reg und Subrout: R4..R11 immer sichern wenn benutzt, R0..R3 und R12 müssen nicht
 Regeln für Stack: wird normalerweise in 8Byte schritten alloziert, muss immer abgebaut(geleert) werden
 Regelung Parameterübergabe Funktionen: Input -> r0-r3 (rest evtl im stack), Return -> in r0 (oder r1, r0 wenn 64bit)
 Reentrant code: kann sich selber aufrufen (rekursive Funktion)
 Unterschiede interrupt(I), Subroutine(s): s->wird bewusst durch ing. aufgerufen, i-> zufällig aufgerufen, ist in vektortabelle abgelegt, keine Parameter oder returns
 Welche Register werden bei Sprung in Interrupt gesichert: r0-r3, LR, PC, xPSR
 Cycles für Sprung in ISR (Interruptserviceroutine): M0 -> 16cycles, M0+ -> 15cycles, M4 -> 12cycles
 TailChaining: Tiefer priorisierter IPO wird während laufendem ISP ->

Brotzessortypen

Cisc= Complex Instruction Set Computer
Risc = Reduced Instruction Set Computer:

- kleine Anzahl Befehle
- Grosse Anzahl verwendbarer Register
- Arbeiten Ausschliesslich mit Registeroperationen
- Schrittweises abbauen von Befehlen

Zu Beachten beim Kaufen:

- Gehäuse Grösse & Aus- Eingänge
- Bitbreite/ Grösse Adressraum
- Spannung/ Leistungsaufnahme
- Systemtakt
- Kosten & Vefügbareit
- Entwicklungs Software

SPEC- Benchmarks: (Marketing)

- Leistungsfähigkeit für die Berechnung (FLOPS, Floating Point Operations per second)
- Anzahl Instruktionen per Zeit (MIPS)
- Takte pro Befehl (in CPI)
- Gut: MIPS/A

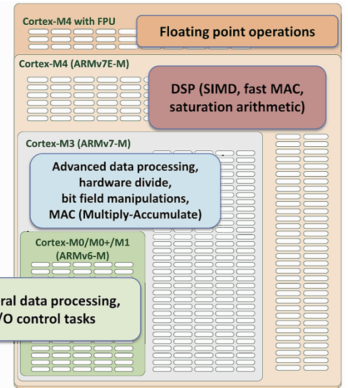
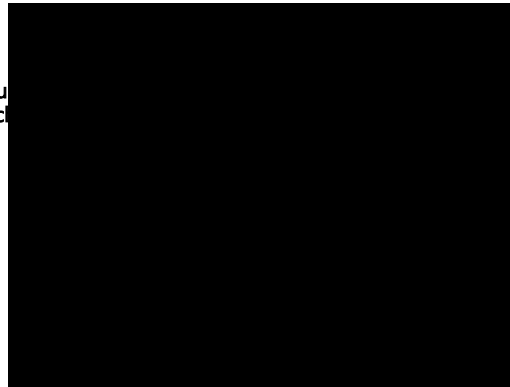
ARM Prozessoren

Types:

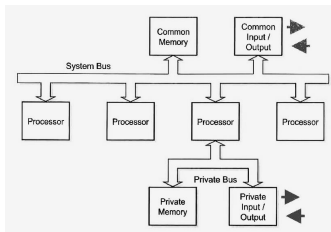
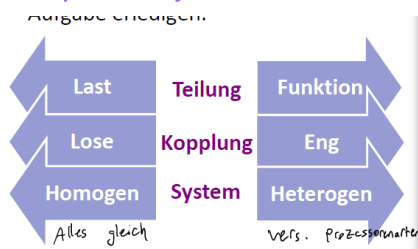
M Mikrocontroller
R Realtime
A Application

Unterschied M0+ zu M0

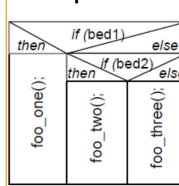
- Zweistufige Pipeline
- MPU Optional erhältlich
- Single Cycles I/O für schnelle Auslesung
- Micro Trace Buffer, zusätzlicher Speicher für Debugbefehle



Multiprozessorsysteme:

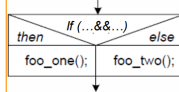


7 Beispiele



Thumb-2 Assembly	C Source Code
<pre> LDR R0, =G2 LDR R1, =G1 LDHR R0, [R0] ;R0 ← G2 SXTB R0, R0 LDR R1, [R1] ;R1 ← G1 CMP R0, R1 ;G2-G1 BGE else_if then: BL foo_one B endif else_if: CMP R0, R1 BLE else BL foo_two B endif else: BL foo_three endif </pre>	<pre> int32_t G1; // signed 32bit int16_t G2; // signed 16bit if(G1 >= G2) // G2-G1 < 0? { foo_one(); } else if(G1 < G2) // G2-G1 > 0? { foo_two(); } else // G2 == G1 { foo_three(); } </pre>

Thumb-2 Assembly	C Source Code
<pre> LDR R0, =G1 LDR R0, [R0] ;R0 ← G1 MOVS R1, #4 ;R1 = 4 CMP R0, R1 ;G1-4 BGT else then: BL foo_one B endif else: BL foo_two endif </pre>	<pre> int32_t G1; // signed 32bit if((G1 > 4) && (G1 <= 4)) { foo_one(); } else { foo_two(); } </pre>



Example: 64Bit Addition

```

LDR r4, =addrOp ;load address of int64 array [op1,op2,res]
LDM r4, {r0-r1} ;load op1[r1,r0] and op2[r2,r3]
ADDS r0, r0, r1 ;add lower words and set carry
ADCS r1, r2, r3 ;add higher words with carry in
ADDS r4, #8 ;set r4 to address of res
STM r4, {r0-r1} ;store 64bit result from r0, r1 to res

```

Example for 64bit logical shift left

```

LDR r2, =data64 ;load address of data64
LDM r2, {r0-r1} ;load data64 to r0, r1
LSLS r1, r1, #1 ;logical shift left r1 by 1
MOVS r3, #0 ;r3 = 0
LSLS r0, r0, #1 ;lsl r0 by 1 (carry = b31)
ADCS r1, r3 ;r1 = r1 + 0 + carry
STM r2!, {r0-r1} ;store r0,r1 in data64

```

R1				R0			
b ₆₃	...	b ₃₃	b ₃₂	b ₃₁	...	b ₁	b ₀
b ₆₂	...	b ₃₂	0	b ₃₁	...	b ₁	b ₀

b ₆₂	...	b ₃₂	0	b ₃₀	...	b ₀	0
b ₆₂	...	b ₃₂	b ₃₁	b ₃₀	...	b ₀	0

Instruktionssatz

ARM, Thumb:

ARM -> 16 Bit Befehle

Thumb -> 32 Bit

Thumb 2 -> 16 & 32 Bit

ARM hat mehr Performance aber benötigt mehr Platz als Thumb 1

Thumb 2 "kombiniert" beide
(Spart Speicherplatz & Vereinfachung komplexer Rechnungen (Division etc.))

Assembler Syntax: (2.3)

- Unified Assembler Language (UAL)
- einheitlicher Syntax für ARM & Thumb

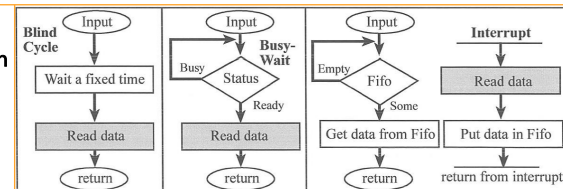
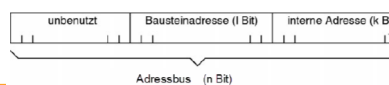
- Label: optionale Adressmarkierung
- OpCode: spezifiziert den Befehl
- Operanden: Parameter des Befehles
- comment: Optionaler Kommentar

I/O Schnittstelle:

Datenregister: zu verarbeitende Daten
Steuerregister: Konfiguration I/O
Statusregister: Status I/O

I/O Adresse: (1.1 Cortex Memory Map)

Jeder Zustand besitzt eigene Adresse



Gegenüberstellung der alternativen Methoden

	Blind Cycle	Busy-Wait	Interrupt
t-Case Latency	Fixed	Unpredictable	Best
Hardware Cost	Least	Low	Moderate
Hardware Complexity	Low	Low	Moderate
I/O Load required	High	High	Minimal

6.1 Interrupts:

Tail Chaining:

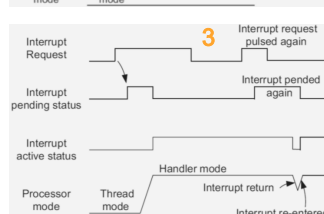
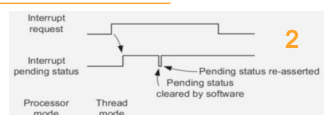
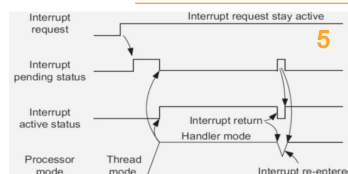
Folgen 2 Interrupts nacheinander wird das Stacking nicht durchgeführt, resp. abgebrochen

Late Arrival:

Kommt ein zweiter, hoher Priorisierter Interrupt während des Stacking vom ersten, wird nachdem Stacking der zweite Interrupt abgearbeitet

POP-Preemption:

Tritt ein Interrupt während dem Unstacking ein, wird dieses unterbrochen und der Interrupt ausgeführt



1. Software Clearing 2. Software Clearing mit Reset 3. ISR Reentry 4. Repulsed IRQ 5. Pulsed IRQ

